

MetroMatrix Social Authentication Integration Guide

Overview

This guide will help you integrate Google and Facebook authentication into your MetroMatrix project. The integration:

-  Uses **native SDKs** for Android/iOS (better UX, no webview)
 -  Uses **expo-auth-session** for Web
 -  Preserves your **existing Redux patterns** and storage
 -  Maintains **admin detection** logic
 -  Works with your **current navigation** structure
-

Prerequisites

Before starting, you need:

1. **Firebase Project** (for authentication backend)
 2. **Google Cloud Console** project (for Google Sign-In)
 3. **Facebook Developer** account (for Facebook Sign-In)
 4. **MetroMatrix Backend** must support these endpoints:
 - `POST /auth/google-login` - Verify Google ID token
 - `POST /auth/facebook-login` - Verify Facebook access token
-

Step 1: Install Dependencies

```
bash  
  
npm install
```

This will install the new dependencies added to `package.json`:

- `@react-native-google-signin/google-signin` - Native Google Sign-In
- `react-native-fbsdk-next` - Native Facebook SDK
- `expo-auth-session` - Web OAuth flows
- `expo-crypto` - Required by expo-auth-session
- `firebase` - Firebase Authentication

- `expo-dev-client` - Required for custom native code
-

Step 2: Firebase Setup

2.1 Create Firebase Project

1. Go to [Firebase Console](#)
2. Click "**Add project**" or select existing project
3. Name it "**MetroMatrix**"
4. Disable Google Analytics (optional)
5. Click "**Create project**"

2.2 Enable Authentication Methods

1. In Firebase Console → **Authentication** → **Sign-in method**
2. Enable **Google** provider
3. Enable **Facebook** provider (you'll configure this later)
4. Enable **Email/Password** (if not already enabled)

2.3 Get Firebase Config

1. Go to **Project Settings** (gear icon)
2. Scroll to "**Your apps**"
3. Click **Web app** icon ()
4. Register app with nickname "MetroMatrix Web"
5. Copy the `firebaseConfig` object
6. Update `firebaseConfig.ts` with your values

2.4 Add Android App to Firebase

1. Still in **Project Settings** → **Your apps**
2. Click **Android** icon
3. Enter package name: `com.metromatrix.app`
4. Download `google-services.json`
5. Place it in **project root**: `RealProject/google-services.json`

2.5 Add iOS App to Firebase

1. Still in **Project Settings** → **Your apps**

2. Click **iOS** icon
 3. Enter bundle ID: `com.metromatrix.app`
 4. Download `GoogleService-Info.plist`
 5. Place it in **project root**: `RealProject/GoogleService-Info.plist`
-

Step 3: Google Cloud Console Setup

3.1 Create OAuth Credentials

1. Go to Google Cloud Console
2. Select your project (or create one)
3. Go to **APIs & Services** → **Credentials**

3.2 Create Web Client ID

1. Click **"Create Credentials"** → **OAuth 2.0 Client ID**
2. Application type: **Web application**
3. Name: "MetroMatrix Web"
4. Authorized redirect URIs:

`https://auth.expo.io/@your-expo-username/MetroMatrix`

5. Click **Create**
6. Copy the **Client ID** (ends with `.apps.googleusercontent.com`)
7. Save this as `GOOGLE_WEB_CLIENT_ID`

3.3 Create Android Client ID

1. Click **"Create Credentials"** → **OAuth 2.0 Client ID**
2. Application type: **Android**
3. Name: "MetroMatrix Android"
4. Package name: `com.metromatrix.app`
5. **SHA-1 certificate fingerprint:**

```
bash
```

Get SHA-1 from EAS

eas credentials

Select Android → Production → View

Copy the SHA-1 fingerprint

6. Click **Create**
7. Copy the **Client ID**
8. Save this as `GOOGLE_ANDROID_CLIENT_ID`

3.4 Create iOS Client ID

1. Click "**Create Credentials**" → **OAuth 2.0 Client ID**
2. Application type: **iOS**
3. Name: "MetroMatrix iOS"
4. Bundle ID: `com.metromatrix.app`
5. Click **Create**
6. Copy the **Client ID**
7. Save this as `GOOGLE_IOS_CLIENT_ID`

3.5 Update authConfig.ts

Open `RealProject/networks/authcalls/authConfig.ts`:

typescript

```
export const AUTH_CONFIG: AuthConfig = {  
  GOOGLE_WEB_CLIENT_ID: 'YOUR_WEB_CLIENT_ID.apps.googleusercontent.com',  
  GOOGLE_ANDROID_CLIENT_ID: 'YOUR_ANDROID_CLIENT_ID.apps.googleusercontent.com',  
  GOOGLE_IOS_CLIENT_ID: 'YOUR_IOS_CLIENT_ID.apps.googleusercontent.com',  
  FACEBOOK_APP_ID: 'YOUR_FACEBOOK_APP_ID',  
};
```

Step 4: Facebook Setup

4.1 Create Facebook App

1. Go to Facebook Developers
2. Click "**My Apps**" → "**Create App**"
3. Use case: **Other** → **Next**
4. App type: **Consumer** → **Next**

5. App name: "**MetroMatrix**"

6. Click "**Create app**"

4.2 Get App Credentials

1. Go to **Settings** → **Basic**
2. Copy **App ID** → Save as `FACEBOOK_APP_ID`
3. Show and copy **App Secret**
4. Click **Show** next to **Client Token** → Save this value

4.3 Add Facebook Login

1. In your app dashboard, click "**Add Product**"
2. Find "**Facebook Login**" → Click "**Set Up**"
3. Select **Android** and **iOS** platforms

4.4 Configure Android Platform

1. Go to **Settings** → **Basic** → **Add Platform** → **Android**
2. Package name: `com.metromatrix.app`
3. Class name: `com.metromatrix.MainActivity`
4. Key hash:

```
bash
```

```
# Convert your SHA-1 to Facebook key hash
```

```
# Use online converter or:
```

```
echo "YOUR_SHA1_FROM_EAS" | xxd -r -p | openssl base64
```

5. Save changes

4.5 Configure iOS Platform

1. Go to **Settings** → **Basic** → **Add Platform** → **iOS**
2. Bundle ID: `com.metromatrix.app`
3. Save changes

4.6 Configure Facebook Login Settings

1. Go to **Facebook Login** → **Settings**
2. Valid OAuth Redirect URIs:

```
https://auth.expo.io/@your-expo-username/MetroMatrix  
fbYOUR_FACEBOOK_APP_ID://authorize
```

3. Save changes

4.7 Make App Public

 **IMPORTANT:** For production use:

1. Go to **Settings** → **Basic**
2. Toggle "**App Mode**" to **Live**
3. This allows any Facebook user to sign in

For development:

- Keep in **Development Mode**
- Add test users in **Roles** → **Test Users**

4.8 Update authConfig.ts

```
typescript  
  
FACEBOOK_APP_ID: 'YOUR_FACEBOOK_APP_ID',
```

4.9 Update app.json

Replace placeholders in `app.json`:

```
json
```

```

{
  "expo": {
    "ios": {
      "infoPlist": {
        "CFBundleURLTypes": [
          {
            "CFBundleURLSchemes": ["fbYOUR_FACEBOOK_APP_ID"]
          }
        ],
        "FacebookAppID": "YOUR_FACEBOOK_APP_ID"
      }
    },
    "plugins": [
      [
        "react-native-fbsdk-next",
        {
          "appID": "YOUR_FACEBOOK_APP_ID",
          "clientToken": "YOUR_FACEBOOK_CLIENT_TOKEN"
        }
      ]
    ]
  }
}

```

Step 5: Backend Integration

Your MetroMatrix backend needs two new endpoints to verify social login tokens:

5.1 POST /auth/google-login

Request:

```

json

{
  "idToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6...",
  "userType": "user"
}

```

Backend Implementation (Node.js example):

```

javascript

```

```
const admin = require('firebase-admin');

app.post('/auth/google-login', async (req, res) => {
  const { idToken, userType } = req.body;

  try {
    // Verify the ID token with Firebase
    const decodedToken = await admin.auth().verifyIdToken(idToken);
    const { uid, email, name, picture } = decodedToken;

    // Check if user exists in your database
    let user = await User.findOne({ email });

    if (!user) {
      // Create new user
      user = await User.create({
        email,
        fullName: name,
        profilePhoto: picture,
        authProvider: 'google',
        googleId: uid,
        isVerified: true, // Google accounts are pre-verified
      });
    }

    // Generate your app's access token
    const accessToken = generateAccessToken(user);
    const refreshToken = generateRefreshToken(user);

    return res.json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        fullName: user.fullName,
        profilePhoto: user.profilePhoto,
        isVerified: user.isVerified,
      },
      accessToken,
      refreshToken,
    });
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Invalid Google token',
    });
  }
});
```

```
}  
});
```

5.2 POST /auth/facebook-login

Request:

```
json  
  
{  
  "accessToken": "EAABsbCS1iHgBO7rZC...",  
  "userType": "user"  
}
```

Backend Implementation:

```
javascript
```

```
const axios = require('axios');

app.post('/auth/facebook-login', async (req, res) => {
  const { accessToken, userType } = req.body;

  try {
    // Verify token with Facebook Graph API
    const fbResponse = await axios.get(
      `https://graph.facebook.com/me?fields=id,name,email,picture&access_token=${accessToken}`
    );

    const { id, name, email, picture } = fbResponse.data;

    if (!email) {
      return res.status(400).json({
        success: false,
        message: 'Email permission required',
      });
    }

    // Check if user exists
    let user = await User.findOne({ email });

    if (!user) {
      // Create new user
      user = await User.create({
        email,
        fullName: name,
        profilePhoto: picture?.data?.url,
        authProvider: 'facebook',
        facebookId: id,
        isVerified: true,
      });
    }

    // Generate tokens
    const accessToken = generateAccessToken(user);
    const refreshToken = generateRefreshToken(user);

    return res.json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        fullName: user.fullName,
        profilePhoto: user.profilePhoto,
      },
    });
  } catch (error) {
    console.error(error);
    return res.status(500).json({ success: false, message: 'Internal server error' });
  }
});
```

```

      isVerified: user.isVerified,
    },
    accessToken,
    refreshToken,
  });
} catch (error) {
  return res.status(401).json({
    success: false,
    message: 'Invalid Facebook token',
  });
}
});

```



Step 6: Configure Google Sign-In SDK

Create a new file: `RealProject/networks/authcalls/googleSignInConfig.ts`

typescript

```

import { GoogleSignin } from '@react-native-google-signin/google-signin';
import { AUTH_CONFIG } from './authConfig';

export const configureGoogleSignIn = () => {
  GoogleSignin.configure({
    webClientId: AUTH_CONFIG.GOOGLE_WEB_CLIENT_ID,
    iosClientId: AUTH_CONFIG.GOOGLE_IOS_CLIENT_ID,
    offlineAccess: true,
    forceCodeForRefreshToken: true,
  });
};

```

Then call this in your `App.tsx`:

typescript

```

import { configureGoogleSignIn } from './networks/authcalls/googleSignInConfig';
import { Platform } from 'react-native';

// In your App component
useEffect(() => {
  if (Platform.OS === 'android' || Platform.OS === 'ios') {
    configureGoogleSignIn();
  }
}, []);

```

Step 7: Testing

Test on Web

```
bash  
  
npm start  
  
# Press 'w' for web
```

1. Click "Sign in with Google" → Should open Google account picker
2. Click "Sign in with Facebook" → Should open Facebook login
3. Check Redux DevTools → User should be saved to state
4. Check AsyncStorage → Tokens should be saved

Test on Android

```
bash  
  
# Build development client  
eas build --profile development --platform android  
  
# Or use Expo Dev Client  
npx expo run:android
```

1. Install APK on device
2. Test Google Sign-In → Should show native account picker (NOT webview!)
3. Test Facebook Sign-In → Should show native Facebook login
4. Verify user is routed to `UserHome` (not `AdminDashboard`)

Test on iOS (when available)

```
bash  
  
eas build --profile development --platform ios  
  
# Or  
npx expo run:ios
```

Step 8: Verify Integration

Check Redux State

After successful login, verify in Redux DevTools:

```
javascript

{
  signIn: {
    user: {
      id: "...",
      email: "user@example.com",
      fullName: "John Doe",
      profilePhoto: "https://...",
      isVerified: true
    },
    accessToken: "eyJhbGciOiJIUzI1NiIs...",
    userType: "user",
    socialLoginStatus: "idle"
  }
}
```

Check AsyncStorage

Verify data is saved:

```
javascript

// Should be saved:
accessToken: "eyJhbGciOiJIUzI1NiIs..."
refreshToken: "eyJhbGciOiJIUzI1NiIs..."
userInfo: "{\"id\":\"...\",\"email\":\"...\"}"
userType: "user"
isAuthenticated: true
```

Check Navigation

- ✅ Admin emails (waleedhassansfd@gmail.com) → AdminDashboard
- ✅ Social logins → UserHome
- ✅ Regular email logins → UserHome

Troubleshooting

Google Sign-In Issues

Problem: "Sign-in failed" or redirects to google.com

Solutions:

1. Check SHA-1 certificate in Google Cloud Console
2. Verify `google-services.json` has correct `certificate_hash`
3. Rebuild app after changes: `eas build`
4. Check Android package name matches: `com.metromatrix.app`

Problem: "No ID token received"

Solutions:

1. Ensure Web Client ID is correct in `authConfig.ts`
2. Check Firebase project has Google auth enabled
3. Verify `GoogleSignin.configure()` is called before sign-in

Facebook Sign-In Issues

Problem: "Invalid scopes" error

Solutions:

1. Use only `public_profile` permission (no app review needed)
2. Make sure you're added as app developer
3. Check app is in Live mode (or you're a test user)

Problem: "App ID not configured"

Solutions:

1. Verify `FACEBOOK_APP_ID` in `authConfig.ts`
2. Check `app.json` has correct Facebook configuration
3. Rebuild app after changes

Backend Issues

Problem: "Invalid token" from backend

Solutions:

1. Verify backend endpoints exist: `/auth/google-login`, `/auth/facebook-login`
2. Check backend is verifying tokens correctly
3. Ensure backend returns proper response format
4. Check network logs for actual error message

Redux/Storage Issues

Problem: User data not saved

Solutions:

1. Check `saveAuthToStorage` is being called
 2. Verify tokens are valid (not null/undefined)
 3. Check AsyncStorage permissions
 4. Look for console errors during save
-

Success Checklist

Before deploying:

- ☐ Firebase project created and configured
 - ☐ Google OAuth credentials created (Web, Android, iOS)
 - ☐ Facebook app created and configured
 - ☐ `firebaseConfig.ts` updated with your values
 - ☐ `authConfig.ts` updated with your credentials
 - ☐ `google-services.json` placed in project root
 - ☐ `GoogleService-Info.plist` placed in project root (for iOS)
 - ☐ `app.json` updated with Facebook App ID
 - ☐ Backend endpoints implemented and tested
 - ☐ Google Sign-In works on Web ✓
 - ☐ Google Sign-In works on Android ✓
 - ☐ Facebook Sign-In works on Web ✓
 - ☐ Facebook Sign-In works on Android ✓
 - ☐ User data saved to Redux ✓
 - ☐ Tokens saved to AsyncStorage ✓
 - ☐ Navigation works correctly ✓
 - ☐ Admin detection works ✓
-

Additional Resources

- [Firebase Setup Guide](#)
- [Google Sign-In for Android](#)
- [Facebook Login Documentation](#)
- [Expo Auth Session](#)



Need Help?

If you encounter issues:

1. Check console logs for detailed error messages
2. Verify all credentials are correct
3. Ensure backend is running and accessible
4. Test on multiple devices/platforms
5. Check that admin email detection works

Integration completed! Your MetroMatrix app now supports Google and Facebook authentication! 🎉